

TD1 - PRAM - Solutions

Organization. The exercises that we cannot cover in class are homework: their solutions will be posted, and we may go over some of them later in class. The algorithms of an exercise may be used in the following exercises.

Conventions. Arrays are indexed from 1, and NIL denotes the null pointer. The *time* of an algorithm is its parallel running time on the PRAM, using the number of PUs it is given. Its *work*, cost, work-optimality and efficiency are defined as in the lecture, with respect to the time T_{seq} of the best sequential algorithm for the same problem. The following algorithms of the lecture may be called as building blocks; they all run on an EREW PRAM with N PUs.

Algorithm	Returns	Time	Work	Cost
BCAST(e, N)	$pe[1 \dots N]$, with $pe[i] = e$ for all i	$O(\log_2 N)$	$O(N)$	$O(N \log_2 N)$
REDUCE($A[1 \dots N], \otimes$)	$A[1] \otimes \dots \otimes A[N]$, for an associative $\otimes$	$O(\log_2 N)$	$O(N)$	$O(N \log_2 N)$
RANKCOMPUTATION($next[1 \dots N]$)	$d[1 \dots N]$, the distances to the end of the list	$O(\log_2 N)$	$O(N \log_2 N)$	$O(N \log_2 N)$

Exercise 1

Is the array sorted?

Let $A[1 \dots N]$ be an array of N integers.

- Design an algorithm ISORTED($A[1 \dots N]$) that returns *true* if $A[1] \leq A[2] \leq \dots \leq A[N]$ and *false* otherwise, in $O(1)$ time on a CRCW PRAM with N PUs. List all the concurrency modes that can be employed for the writes of this algorithm.
- Verify the reads of your algorithm, by decomposing them into elementary steps as was done for list ranking in the lecture: are they exclusive? Does the algorithm need concurrent reads, or only concurrent writes?
- Design an EREW PRAM version of ISORTED with N PUs. What are its time, work and cost? Is it work-optimal? Is it efficient?

Solution

(a) One PU per pair of consecutive elements compares them, and reports a violation into a shared flag.

Every PU that writes *sorted* writes the same value *false*, so every mode works: the **consistent** mode is enough, and the **arbitrary** and **priority** modes give the same result, as all the written values are equal. In **fusion mode**, every PU i writes its own verdict $sorted \leftarrow (A[i] \leq A[i+1])$ instead, with **and** as the operation (or **min**, with $false = 0$ and $true = 1$). Time $O(1)$ with $N - 1$ PUs (line 3 is a single comparison), work $O(N)$ and cost $O(N)$: as $T_{seq}(N) = O(N)$, the algorithm is work-optimal and efficient.

(b) The cell $A[i+1]$ is read by two PUs, i and $i+1$, but not at the same step. Line 3 decomposes into: $temp1 \leftarrow \text{LOAD}(A+i)$, $temp2 \leftarrow \text{LOAD}(A+i+1)$, compare, store. All PUs execute the same elementary step at the same time: in the first step PU i reads $A[i]$, every PU a different cell; in the second step PU i reads $A[i+1]$, again every PU a different cell ($i \mapsto i+1$ is injective). All reads are exclusive: the algorithm needs concurrent writes only (the writes of *false* into *sorted*).

(c) Without concurrent writes, every PU writes its verdict into its own cell, and the verdicts are combined by a reduction with **and**.

Line 2 takes $O(1)$ time and $O(N)$ work, exclusive accesses as in (b); REDUCE takes $O(\log_2 N)$ time and $O(N)$ work on an EREW PRAM. Time $T(N) = O(\log N)$, work $W(N) = O(N)$, cost $C(N) = N \cdot O(\log N) = O(N \log N)$. Since $T_{seq}(N) = O(N)$: work-optimal, but not efficient (the profile of REDUCE: the $\log N$ factor of the cost is paid by idle PUs).

ISORTED($A[1 \dots N]$)

```

1: sorted ← true
2: forall i = 1, ..., N - 1 in parallel do
3:   if A[i] > A[i + 1] then sorted ← false
4: return sorted

```

ISORTEDEREW($A[1 \dots N]$)

```

1: forall i = 1, ..., N in parallel do
2:   ok[i] ← (i = N or A[i] ≤ A[i + 1])
3: return REDUCE(ok[1 ... N], and)

```

Exercise 2

From a list to a ring

Let $next[1 \dots N]$ be a linked list of N nodes: $next[i]$ is the successor of node i , and $next[\ell] = \text{NIL}$ for the last node ℓ . The head h of the list is the only node that no other node points to. We want to close the list into a ring, by making the last node point to the head: $next[\ell] \leftarrow h$.

- Design an EREW PRAM algorithm $\text{LISTTORING}(next[1 \dots N])$ with N PUs that updates $next[1 \dots N]$ into a ring and returns the index h of the head. Verify that all reads and writes are exclusive.
- What are its time, work and cost? Is it work-optimal? Is it efficient?

Solution

(a) The head is the only node that is not pointed to: every node marks its successor, and the unmarked node is the head.

Exclusivity: in line 4, two PUs never write the same cell $mark[next[i]]$ because $next$ is injective (two nodes have different successors); in line 6 exactly one PU writes h (the head is unique); in line 8 exactly one PU reads h (the last node is unique) and writes its own cell. The other accesses are on cells of the PU itself. Note that h must be written and read in two different parallel loops (lines 6 and 8), not in the same one.

 $\text{LISTTORING}(next[1 \dots N])$

```

1: forall  $i = 1, \dots, N$  in parallel do
2:    $mark[i] \leftarrow 0$ 
3: forall  $i = 1, \dots, N$  in parallel do
4:   if  $next[i] \neq \text{NIL}$  then  $mark[next[i]] \leftarrow 1$ 
5: forall  $i = 1, \dots, N$  in parallel do
6:   if  $mark[i] = 0$  then  $h \leftarrow i$ 
7: forall  $i = 1, \dots, N$  in parallel do
8:   if  $next[i] = \text{NIL}$  then  $next[i] \leftarrow h$ 
9: return  $h$ 

```

(b) Four parallel loops of $O(1)$ time each: $T(N) = O(1)$ with N PUs, $W(N) = O(N)$, $C(N) = O(N)$. Sequentially, finding the head requires looking at every pointer: $T_{seq}(N) = O(N)$. The algorithm is work-optimal and efficient.

Exercise 3

Inverting a list

Let $next[1 \dots N]$ be a linked list of N nodes.

- Design an EREW PRAM algorithm $\text{INVERT}(next[1 \dots N])$ with N PUs that computes the inverse of this linked list, and returns it in a pointer array $prev[1 \dots N]$ (e.g., if $next[3] = 7$, then $prev[7] = 3$).
- What are its time, work and cost? Is it work-optimal? Is it efficient?
- Now let $next[1 \dots N]$ be a ring (no pointer is NIL), and h the index of one of its nodes. Design an EREW PRAM algorithm $\text{RINGTOLIST}(next[1 \dots N], h)$ with N PUs that turns the ring into a list whose head is h . What is its time?

Solution

(a) Every node writes itself into the $prev$ cell of its successor.

The writes of line 4 are exclusive because $next$ is injective: two different nodes have two different successors, hence write two different cells. Line 2 gives $prev[h] = \text{NIL}$ for the head h , the only node that nobody writes.

 $\text{INVERT}(next[1 \dots N])$

```

1: forall  $i = 1, \dots, N$  in parallel do
2:    $prev[i] \leftarrow \text{NIL}$ 
3: forall  $i = 1, \dots, N$  in parallel do
4:   if  $next[i] \neq \text{NIL}$  then  $prev[next[i]] \leftarrow i$ 
5: return  $prev[1 \dots N]$ 

```

(b) Two parallel loops of $O(1)$ time: $T(N) = O(1)$ with N PUs, $W(N) = O(N)$ and $C(N) = O(N)$. Sequentially, every pointer must be read once: $T_{seq}(N) = O(N)$, so the algorithm is work-optimal and efficient.

(c) The predecessor p of h must be cut: $next[p] \leftarrow \text{NIL}$.

Testing $next[i] = h$ in every PU would make all PUs read h at once, a concurrent read (fixing it with $BCAST(h, N)$ would cost $O(\log N)$). Instead, invert the ring with (a): every node writes itself as the predecessor of its successor (exclusive), then a single PU reads $prev[h]$ and cuts the pointer. Two steps of $O(1)$: time $O(1)$ (work and cost $O(N)$).

This algorithm is used in the next exercises whenever a list must be traversed in the other direction.

RINGTOLIST($next[1 \dots N], h$)

```
1:  $prev[1 \dots N] \leftarrow \text{INVERT}(next[1 \dots N])$  ► (a)
2:  $next[prev[h]] \leftarrow \text{NIL}$  ► one PU
3: return  $next[1 \dots N]$ 
```

Exercise 4

Work-optimal and efficient reduction

Let $A[1 \dots N]$ be an array and $\otimes$ an associative operation. In the lecture, $\text{REDUCE}(A[1 \dots N], \otimes)$ computed $A[1] \otimes \dots \otimes A[N]$ in $O(\log_2 N)$ time on an EREW PRAM, whereas $T_{seq}(N) = O(N)$.

- Write a *recursive* version of the reduction that uses up to N PUs and runs in $O(\log_2 N)$ time, and is work-optimal. Is it efficient?
- Write a non-recursive (*iterative*) version of the reduction that uses up to N PUs, runs in $O(\log_2 N)$ time, and is work-optimal. Is it efficient?
- Design $\text{REDUCEP}(A[1 \dots N], \otimes, P)$ that computes the same reduction with P PUs, $P \ll N$. What are its time, work and cost, as functions of N and P ? For which values of P does the algorithm stay efficient?

Solution

(a) The recursion of the lecture: combine the elements two by two, then reduce the half-size array.

Only the $\lceil n/2 \rceil$ PUs that have something to do are created at each level (line 2): $W(N) \leq \lceil N/2 \rceil + \lceil N/4 \rceil + \dots \leq N + \lceil \log_2 N \rceil = O(N)$, work-optimal. Accesses: PU i reads $A[2i-1]$, $A[2i]$ and writes $A[i]$, all the reads of a line before its writes: exclusive. Time: $T(N) = T(\lceil N/2 \rceil) + O(1) = O(\log N)$, with at most $\lceil N/2 \rceil$ PUs created at any level. Efficient? No: on a machine with N PUs, $C(N) = N \cdot O(\log N) = O(N \log N)$, while $T_{seq}(N) = O(N)$; most PUs are not created most of the time, but the cost pays for them anyway.

(b) The same halving, written as a loop on the current size n of the array.

Written like $BCAST$, with **forall** $i = 1, \dots, N$ and **if** $i \leq \lceil n/2 \rceil$, every iteration would create N PUs and the work would be $O(N \log N)$. Creating only the $\lceil n/2 \rceil$ PUs that combine a pair (line 3), plus one PU for the odd element (line 5), gives $W(N) \leq N/2 + N/4 + \dots + \lceil \log_2 N \rceil = O(N)$: work-optimal, exactly like (a), which creates the same PUs. Accesses: as in (a), exclusive. Time: $\lceil \log_2 N \rceil$ iterations of $O(1)$, $T(N) = O(\log N)$ with at most $N/2$ PUs created; cost $O(N \log N)$ on a machine with N PUs: not efficient, like (a).

(c) The remedy to the cost: cut the array into P blocks of $B = N/P$ consecutive elements (we assume that P divides N ; otherwise the last block is shorter); PU j reduces its block sequentially into $S[j]$, then the P partial results are reduced in parallel with the algorithm of (a).

REDUCE($A[1 \dots n], \otimes$)

```
1: if  $n = 1$  then return  $A[1]$ 
2: forall  $i = 1, \dots, \lceil n/2 \rceil$  in parallel do
3:   if  $2i \leq n$  then
4:      $A[i] \leftarrow A[2i-1] \otimes A[2i]$ 
5:   else
6:      $A[i] \leftarrow A[2i-1]$ 
7: return  $\text{REDUCE}(A[1 \dots \lceil n/2 \rceil], \otimes)$ 
```

REDUCEITER($A[1 \dots N], \otimes$)

```
1:  $n \leftarrow N$ 
2: while  $n > 1$  do
3:   forall  $i = 1, \dots, \lceil n/2 \rceil$  in parallel do
4:      $A[i] \leftarrow A[2i-1] \otimes A[2i]$ 
5:   if  $n$  is odd then  $A[\lceil n/2 \rceil] \leftarrow A[n]$  ► one PU
6:    $n \leftarrow \lceil n/2 \rceil$ 
7: return  $A[1]$ 
```

Exclusivity: in lines 2–5, PU j reads the cells of its own block and writes $S[j]$; line 6 is the EREW algorithm of (a). Time: $O(N/P)$ for the blocks, then $O(\log_2 P)$ for the reduction of $S[1 \dots P]$ with P PUs: $T(N, P) = O(N/P + \log_2 P)$. Work: $N - P$ applications of $\otimes$ in the blocks and $O(P)$ in REDUCE: $W(N, P) = O(N + P) = O(N)$ since $P \leq N$: work-optimal for every P . Cost: $C(N, P) = P \cdot T(N, P) = O(N + P \log_2 P)$: efficient as long as $P \log_2 P = O(N)$, i.e., $P = O(N/\log_2 N)$. With $P = N/\log_2 N$: $N/P = \log_2 N$ and $\log_2 P \leq \log_2 N$, so $T = O(\log N)$, the same time as with N PUs, but $C = O(N)$: work-optimal *and* efficient. The $\log N$ factor of the cost was the price of N PUs, not of the parallel time: $N/\log_2 N$ PUs give the same time. With more PUs ($P = N/\log_2 \log_2 N$, say) the cost grows again, since the $\log_2 P$ steps of line 6 are paid by P PUs.

REDUCEP($A[1 \dots N], \otimes, P$)

```

1:  $B \leftarrow N/P$ 
2: forall  $j = 1, \dots, P$  in parallel do
3:    $S[j] \leftarrow A[(j-1)B + 1]$ 
4:   for  $i = (j-1)B + 2, \dots, jB$  do
5:      $S[j] \leftarrow S[j] \otimes A[i]$ 
6: return REDUCE( $S[1 \dots P], \otimes$ )

```

► block j
► P PUs

Exercise 5

Prefix scan on an array

Let $X[1 \dots N]$ be an array of size N and $\otimes$ an associative operation.

- Design an EREW PRAM algorithm $\text{SCAN}(X[1 \dots N], \otimes)$ that computes and returns the prefix scan array $X'[i] = X[1] \otimes \dots \otimes X[i]$ for all i , using N PUs and running in $O(\log_2 N)$ time. Verify that all reads and writes are exclusive. Hint: Try using recursion on the array. Compute the work and cost of this algorithm. Is it work-optimal? Is it efficient?
- This time, design an algorithm specifically for P PUs that does the same computation in $O(N/P + \log_2 P)$ time, then compute its work and cost. Is it work-optimal? For what values of P does it stay efficient?

Solution

(a) Recursion on the array: combine the elements two by two, compute the prefix scan of the $\lfloor N/2 \rfloor$ pairs recursively, then deduce the prefixes of the original array. If $Z[1 \dots \lfloor N/2 \rfloor]$ is the prefix scan of the pairs, then $Z[k] = X[1] \otimes \dots \otimes X[2k]$: the prefix of an even index $2k$ is $Z[k]$, and the prefix of an odd index $2k + 1$ is $Z[k] \otimes X[2k + 1]$.

Exclusivity: in line 3, PU i reads $X[2i - 1]$ and $X[2i]$ and writes $Y[i]$, all different from one PU to another; in line 6, PU i reads $Z[i]$ and writes $X'[2i]$; in line 9, PU i reads $Z[i - 1]$ and $X[2i - 1]$ and writes $X'[2i - 1]$. The cell $Z[k]$ is needed by two indices, $2k$ (line 6) and $2k + 1$ (line 9): the two loops are kept separate so that each cell is read once per loop.

Time: $T(N) = T(N/2) + O(1)$, hence $T(N) = O(\log_2 N)$, with N PUs at the first level, $N/2$ at the second, $\dots$ (the same N PUs are reused). Work: $W(N) = W(N/2) + O(N)$, hence $W(N) = O(N + N/2 + N/4 + \dots) = O(N)$. Cost: $C(N) = N \cdot O(\log N) = O(N \log N)$. With $T_{\text{seq}}(N) = O(N)$ (one pass over the array), the algorithm is work-optimal but not efficient. Compare with pointer jumping on the list $1 \rightarrow 2 \rightarrow \dots \rightarrow N$ (Exercise 6), which would also take $O(\log N)$ time, but $O(N \log N)$ work.

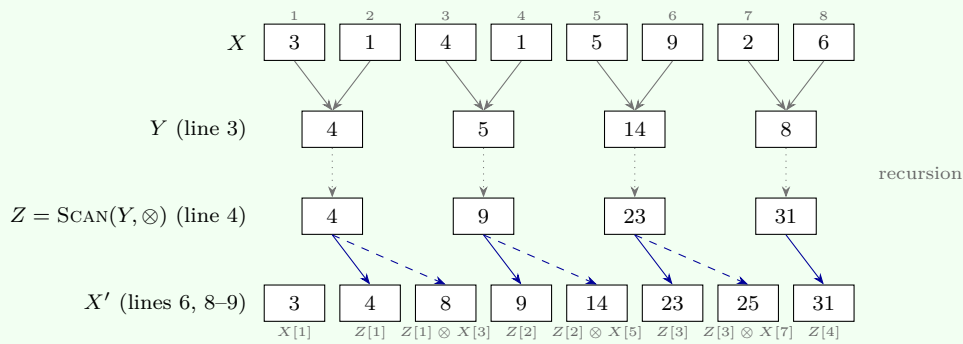
Example with $\otimes = +$ and $N = 8$: the pairs are summed into Y , the recursive call returns Z , the prefix scan of Y , and X' is read off Z : the even positions are copied from Z (line 6), the odd ones add one more element of X (line 9).

SCAN($X[1 \dots N], \otimes$)

```

1: if  $N = 1$  then return  $X[1 \dots 1]$ 
2: forall  $i = 1, \dots, \lfloor N/2 \rfloor$  in parallel do
3:    $Y[i] \leftarrow X[2i - 1] \otimes X[2i]$ 
4:  $Z[1 \dots \lfloor N/2 \rfloor] \leftarrow \text{SCAN}(Y[1 \dots \lfloor N/2 \rfloor], \otimes)$ 
5: forall  $i = 1, \dots, \lfloor N/2 \rfloor$  in parallel do
6:    $X'[2i] \leftarrow Z[i]$ 
7: forall  $i = 1, \dots, \lfloor N/2 \rfloor$  in parallel do
8:   if  $i = 1$  then  $X'[1] \leftarrow X[1]$ 
9:   else  $X'[2i - 1] \leftarrow Z[i - 1] \otimes X[2i - 1]$ 
10: return  $X'[1 \dots N]$ 

```



(b) With P PUs, cut the array into P blocks of $B = N/P$ consecutive elements (we assume that P divides N ; otherwise the last block is shorter). Each PU scans its own block sequentially, the P block totals are scanned in parallel with the algorithm of (a), and each PU finally applies the total of the blocks before its own to its block.

All accesses are exclusive: in lines 2–6 and 8–10, PU j touches only the cells of its block and $S[j]$, $S'[j-1]$; line 7 is the EREW algorithm of (a). Note the order of the operands in line 10: $S'[j-1]$, the combination of all the elements *before* the block, goes on the left. Time: $O(N/P)$ for lines 1–6, $O(\log_2 P)$ for line 7 with P PUs, $O(N/P)$ for lines 8–10: $T(N, P) = O(N/P + \log_2 P)$. Work: $O(N) + O(P) + O(N) = O(N)$: work-optimal for every P . Cost: $C(N, P) = P \cdot T(N, P) = O(N + P \log_2 P)$: efficient as long as $P \log_2 P = O(N)$, i.e., $P = O(N/\log_2 N)$. With $P = N/\log_2 N$ PUs, the time is still $O(\log_2 N)$, as in (a), but with $\log_2 N$ times fewer PUs, the cost drops to $O(N)$: the algorithm becomes efficient.

SCANP($X[1 \dots N], \otimes, P$)

```

1:  $B \leftarrow N/P$ 
2: forall  $j = 1, \dots, P$  in parallel do
3:    $X'[(j-1)B + 1] \leftarrow X[(j-1)B + 1]$ 
4:   for  $i = (j-1)B + 2, \dots, jB$  do
5:      $X'[i] \leftarrow X'[i-1] \otimes X[i]$ 
6:    $S[j] \leftarrow X'[jB]$ 
7:  $S'[1 \dots P] \leftarrow \text{SCAN}(S[1 \dots P], \otimes)$ 
8: forall  $j = 2, \dots, P$  in parallel do
9:   for  $i = (j-1)B + 1, \dots, jB$  do
10:     $X'[i] \leftarrow S'[j-1] \otimes X'[i]$ 
11: return  $X'[1 \dots N]$ 

```

► scan of block j
 ► total of block j
 ► (a), P PUs

Exercise 6

Prefix scan on a list

Let $next[1 \dots N]$ be a linked list of N nodes, with head h and tail t ($next[t] = \text{NIL}$), $x[1 \dots N]$ a value attached to each node, and $\otimes$ an associative operation. In the lecture, the list ranking algorithm computed, with pointer jumping, the distance $d[i]$ from every node i to the end of the list.

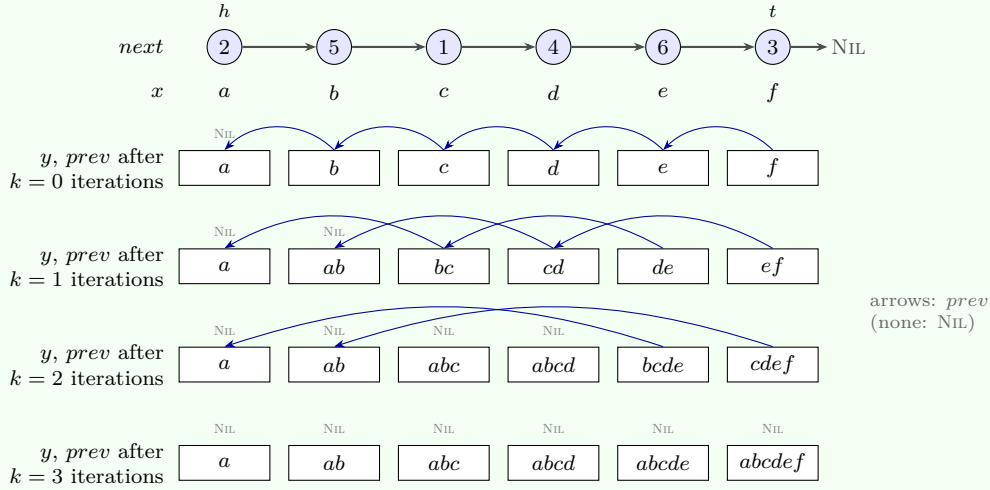
- Design an algorithm $\text{SCAN}(next[1 \dots N], x[1 \dots N], \otimes)$ that computes and returns the array $y[1 \dots N]$ where $y[i] = x[h] \otimes \dots \otimes x[i]$ is the combination of the values from the head h of the list to node i , in $O(\log_2 N)$ time using N PUs. Hint: start from the list ranking algorithm of the lecture.
- Verify the memory accesses of your algorithm: are all reads exclusive? all writes? On which PRAM model does it run as written, and how would you modify it to run on an EREW PRAM?
- Compute the work and cost of this algorithm. Is it work-optimal? Is it efficient?
- Show that computing the rank $r[i]$ of every node from the head of the list (0 for the head, $N-1$ for the tail) is a particular case of SCAN: for which values $x[i]$ and which operation $\otimes$?
- Using a single call to SCAN, design an algorithm $\text{RANKBYPREFIX}(next[1 \dots N])$ with N PUs that computes and returns the distance $d[i]$ from every node to the end of the list.
- Design an algorithm $\text{REVERSESCAN}(next[1 \dots N], x[1 \dots N], \otimes)$ that computes, in the same time, the array $y[1 \dots N]$ where $y[i] = x[i] \otimes \dots \otimes x[t]$ is the combination of the values from node i to the tail t of the list. Pay attention to the order of the operands if $\otimes$ is not commutative.

Solution

(a) The pointer jumping of RANKCOMPUTATION accumulates, from every node, the values of the nodes *after* it; to accumulate the values from the head, jump *towards the head*: invert the list with INVERT (Exercise 3), then jump along $prev$, each node accumulating the value of the node it jumps over, which is on its left.

Invariant: $y[i]$ is the combination of the values from the node just after $prev[i]$ to i (from h if $prev[i] = \text{NIL}$). It holds after line 3, and line 7 preserves it: $y[prev[i]] \otimes y[i]$ covers the nodes from the node after $prev[prev[i]]$ to i . The order of the operands matters when $\otimes$ is not commutative: the node jumped over is *before* i in the list, so its value goes on the left. As in the lecture, after $\lceil \log_2 N \rceil$ iterations every pointer $prev[i]$ is NIL and $y[i] = x[h] \otimes \dots \otimes x[i]$. The list $next$ is left untouched: the jumps destroy $prev$ only.

Example with $\otimes$ the concatenation (abc stands for $a \otimes b \otimes c$) on the list $2 \rightarrow 5 \rightarrow 1 \rightarrow 4 \rightarrow 6 \rightarrow 3$: after k iterations, $y[i]$ combines the values of the 2^k nodes ending at i (fewer near the head), and $prev[i]$ points 2^k nodes back.



(b) Line 1 is exclusive (Exercise 3), and lines 7–8 are the lines 6–7 of RANKCOMPUTATION, with $prev$ instead of $next$ and y instead of d : decomposed into elementary steps, PU i reads its own cells $y[i]$ and $prev[i]$, then the cells $y[prev[i]]$ and $prev[prev[i]]$ of its predecessor, which no other PU reads at the same step since $prev$ is injective (and stays so after a jump: if $prev[prev[i]] = prev[prev[j]]$ then $i = j$); it writes its own cells only. The body of the loop is therefore fine on an EREW PRAM. What depends on the model is the test of line 4, “there exists a node i with $prev[i] \neq \text{NIL}$ ”: as written, every PU writes $done \leftarrow (prev[i] = \text{NIL})$ into a shared flag, in **fusion mode with and**, which requires a CRCW PRAM. On a CREW PRAM, every PU writes its bit into its own cell $pd[i]$ and the flag is computed by $done \leftarrow \text{REDUCE}(pd[1 \dots N], \text{and})$; on an EREW PRAM, the flag must in addition be distributed by $\text{BCAST}(done, N)$ before the PUs test it. The test then costs $O(\log N)$ per iteration, hence $O(\log^2 N)$ in total; as in the lecture, performing it only every $\lceil \log_2 N \rceil$ iterations (or simply running the loop $\lceil \log_2 N \rceil$ times without any test) keeps the total time $O(\log N)$.

(c) Line 1 takes $O(1)$ time and $O(N)$ work; then $\lceil \log_2 N \rceil$ iterations, each creating N PUs that execute an $O(1)$ body (a node whose pointer is already NIL still evaluates the test of line 6): $W(N) = O(N \log N)$ and $C(N) = O(N \log N)$, as for RANKCOMPUTATION. Creating fewer PUs would not help: at least half of the nodes are at distance $r[i] \geq N/2$ from the head, and jump during $\Omega(\log N)$ iterations. With $T_{seq}(N) = O(N)$ (one walk along the list), the algorithm is neither work-optimal nor efficient: pointer jumping trades a $\log N$ factor of work for an exponential gain in time.

(d) Take $\otimes = +$ and $x[i] = 1$ for every node: $y[i]$ is the number of nodes from h to i included, i.e., $y[i] = r[i] + 1$. To obtain $r[i]$ exactly, take $x[h] = 0$ for the head (found as in Exercise 2) and $x[i] = 1$ for the other nodes.

(e) The rank $r[i]$ from the head and the distance $d[i]$ to the end satisfy $r[i] + d[i] = N - 1$.

With $x[i] = 1$ for every node and $\otimes = +$, SCAN returns $y[i] = r[i] + 1 = N - d[i]$. Lines 1–2 and 4–5 take $O(1)$ time and $O(N)$ work: the profile is that of SCAN, $O(\log N)$ time, $O(N \log N)$ work and cost, on the same PRAM model (the pointer jumping of the lecture does the same work, directly along $next$).

SCAN($next[1 \dots N], x[1 \dots N], \otimes$)

```

1:  $prev[1 \dots N] \leftarrow \text{INVERT}(next[1 \dots N])$    ▶ Exercise 3
2: forall  $i = 1, \dots, N$  in parallel do
3:    $y[i] \leftarrow x[i]$ 
4:   while there exists a node  $i$  such that  $prev[i] \neq \text{NIL}$  do
5:     forall  $i = 1, \dots, N$  in parallel do
6:       if  $prev[i] \neq \text{NIL}$  then
7:          $y[i] \leftarrow y[prev[i]] \otimes y[i]$ 
8:          $prev[i] \leftarrow prev[prev[i]]$ 
9:   return  $y[1 \dots N]$ 

```

RANKBYPREFIX($next[1 \dots N]$)

```

1: forall  $i = 1, \dots, N$  in parallel do
2:    $x[i] \leftarrow 1$ 
3:    $y[1 \dots N] \leftarrow \text{SCAN}(next[1 \dots N], x[1 \dots N], +)$ 
4:   forall  $i = 1, \dots, N$  in parallel do
5:      $d[i] \leftarrow N - y[i]$ 
6:   return  $d[1 \dots N]$ 

```

(f) From every node towards the tail, the jumps follow $next$ itself, without inversion, and the node jumped over is now *after* i : its value goes on the right.

Invariant: $y[i]$ is the combination of the values from i to the node just before $next[i]$ (to t if $next[i] = \text{NIL}$); it holds after line 2, line 6 preserves it, and after $\lceil \log_2 N \rceil$ iterations $y[i] = x[i] \otimes \dots \otimes x[t]$: this is exactly RANKCOMPUTATION with $\otimes$ instead of $+$. Same time, work, cost and PRAM models as SCAN, with the same termination test as in (b). This time the jumps overwrite $next$ itself: REVERSESCAN destroys its list, and must run on a copy of $next$ if the list is needed afterwards (as in the next exercise). With $\otimes = +$ and $x[i] = 1$ ($x[t] = 0$), REVERSESCAN computes the distances $d[i]$ directly, without the subtraction of RANKBYPREFIX: it is the list ranking algorithm of the lecture.

REVERSESCAN($next[1 \dots N], x[1 \dots N], \otimes$)

```

1: forall  $i = 1, \dots, N$  in parallel do
2:    $y[i] \leftarrow x[i]$ 
3:   while there exists a node  $i$  such that  $next[i] \neq \text{NIL}$  do
4:     forall  $i = 1, \dots, N$  in parallel do
5:       if  $next[i] \neq \text{NIL}$  then
6:          $y[i] \leftarrow y[i] \otimes y[next[i]]$ 
7:          $next[i] \leftarrow next[next[i]]$ 
8:   return  $y[1 \dots N]$ 

```

Exercise 7

Selection in a list on an EREW PRAM

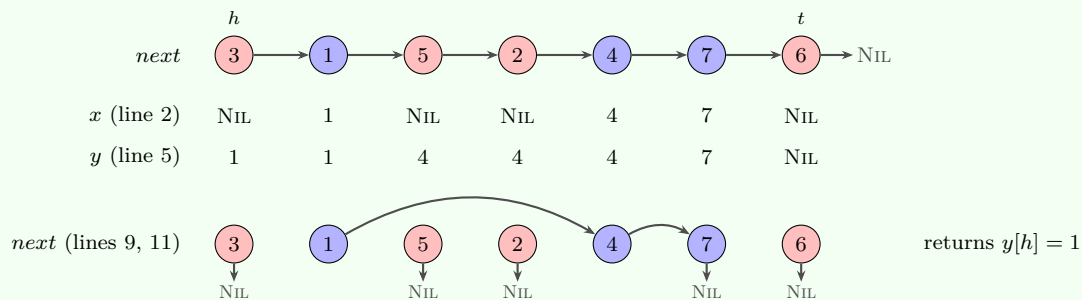
Let $next[1 \dots N]$ be a linked list of N nodes, each node i having a label $label[i] \in \{\text{red}, \text{blue}\}$.

- Design an EREW PRAM algorithm SELECTBLUES($next[1 \dots N], label[1 \dots N]$) with N PUs that updates the list $next[1 \dots N]$ so that in the end i) each blue node points to the next blue node in the original list, ii) the last blue node and all the red nodes point to NIL, and iii) the algorithm returns the index of the first blue node, the head of the updated list of blue nodes (NIL if there is no blue node).
- What are its time, work and cost?

Solution

(a) Every node must find the *first blue node at or after itself* in the list: this is a REVERSESCAN (Exercise 6(f)) with a well-chosen operation, in three steps.

- Mark the nodes: $x[i] = i$ if node i is blue, $x[i] = \text{NIL}$ otherwise (line 2).
- Combine the marks with $a \otimes b = a$ if $a \neq \text{NIL}$, b otherwise (“the first non-NIL value”), which is associative: $y[i] = x[i] \otimes x[next[i]] \otimes \dots \otimes x[t]$ is the first blue node from i included, or NIL if there is none (line 5).
- Relink: a blue node i points to $y[next[i]]$, the first blue node after it, which is NIL for the last blue node (all its successors are red, or it is the tail); the red nodes point to NIL (lines 7–11). The head of the blue list is $y[h]$, the first blue node from the head h of the list, NIL if there is no blue node (line 12).



REVERSESCAN destroys its list and *next* is still needed in line 9: it runs on a copy (lines 3–5). Exclusivity: line 9 reads $y[\text{next}[i]]$, a cell that no other PU reads at the same step since *next* is injective; lines 2, 4 and 11 touch the cells of PU i only; the head is found as in Exercise 2 and read once; REVERSESCAN runs on an EREW PRAM with the termination test of Exercise 6(b).

(b) Lines 1–4 and 6–12 take $O(1)$ time and $O(N)$ work; line 5 takes $O(\log N)$ time and $O(N \log N)$ work with N PUs. Time $O(\log N)$, work $O(N \log N)$, cost $O(N \log N)$: those of REVERSESCAN, neither work-optimal nor efficient, with $T_{seq}(N) = O(N)$.

SELECTBLUES(*next*[1... N], *label*[1... N])

```

1: forall  $i = 1, \dots, N$  in parallel do
2:   if label[ $i$ ] = blue then  $x[i] \leftarrow i$  else  $x[i] \leftarrow \text{NIL}$ 
3: forall  $i = 1, \dots, N$  in parallel do
4:    $\text{copy}[i] \leftarrow \text{next}[i]$ 
5:  $y[1 \dots N] \leftarrow \text{REVERSESCAN}(\text{copy}[1 \dots N], x[1 \dots N], \otimes)$ 
6:  $h \leftarrow \text{head of the list}$  ► Exercise 2, lines 1–6
7: forall  $i = 1, \dots, N$  in parallel do
8:   if label[ $i$ ] = blue and next[ $i$ ]  $\neq \text{NIL}$  then ► next blue
9:      $\text{next}[i] \leftarrow y[\text{next}[i]]$ 
10:  else
11:     $\text{next}[i] \leftarrow \text{NIL}$ 
12: return  $y[h]$ 

```

Exercise 8

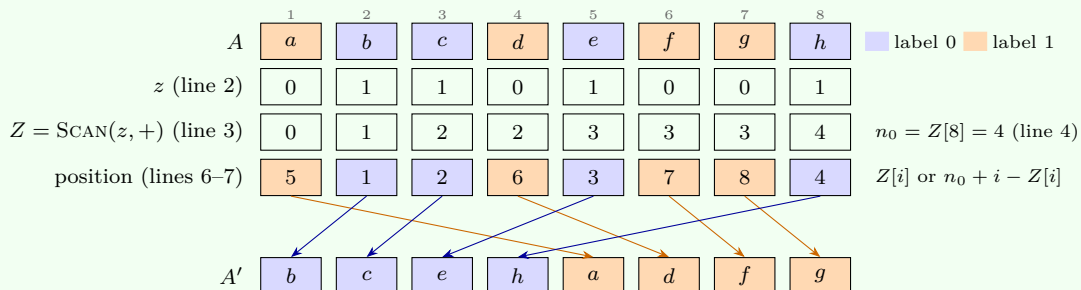
Separation in an array on an EREW PRAM

Let $A[1 \dots N]$ be an array of N objects, each element having a label $\text{LABEL}(A[i]) \in \{0, 1\}$.

- Design an EREW PRAM algorithm SEPARATE01($A[1 \dots N]$) with N PUs that returns a copy $A'[1 \dots N]$ of A permuted so that all the elements of label 0 come first (the labels of $A'[1], \dots, A'[N]$ read $0, \dots, 0, 1, \dots, 1$). Hint: Try to use the prefix scan algorithm to determine where to write each element after permutation.
- What are its time, work and cost?

Solution

(a) An element with label 0 at index i goes to position $Z[i]$, the number of elements with label 0 among $A[1 \dots i]$: a prefix scan with $+$ of the indicator of label 0. An element with label 1 at index i goes after all the n_0 elements with label 0, at position $n_0 + (i - Z[i])$, since $i - Z[i]$ is the number of elements with label 1 among $A[1 \dots i]$.



The total $n_0 = Z[N]$ is needed by every PU: reading it directly would be a concurrent read, so it is distributed with BCAST (line 4). Writes: the positions $Z[i]$ of the elements with label 0 are distinct (as Z increases by 1 at each of them), and so are the positions $n_0 + i - Z[i]$ of the elements with label 1; the two groups of positions are $1 \dots n_0$ and $n_0 + 1 \dots N$. All the other accesses are on the cells of PU i : EREW. The relative order of the elements is preserved within each group (a stable separation).

(b) Lines 1–2 and 5–7 take $O(1)$ time and $O(N)$ work; SCAN (Exercise 5(a)) and BCAST take $O(\log N)$ time and $O(N)$ work each. Time $O(\log N)$, work $O(N)$, cost $O(N \log N)$: work-optimal, not efficient, with $T_{seq}(N) = O(N)$.

SEPARATE01($A[1 \dots N]$)

```

1: forall  $i = 1, \dots, N$  in parallel do
2:   if LABEL( $A[i]$ ) = 0 then  $z[i] \leftarrow 1$  else  $z[i] \leftarrow 0$ 
3:  $Z[1 \dots N] \leftarrow \text{SCAN}(z[1 \dots N], +)$  ► Exercise 5
4:  $pn_0[1 \dots N] \leftarrow \text{BCAST}(Z[N], N)$ 
5: forall  $i = 1, \dots, N$  in parallel do
6:   if LABEL( $A[i]$ ) = 0 then  $A'[Z[i]] \leftarrow A[i]$ 
7:   else  $A'[pn_0[i] + i - Z[i]] \leftarrow A[i]$ 
8: return  $A'[1 \dots N]$ 

```

Exercise 9

Sorting an array of unique integers

Let $X[1 \dots N]$ be an array of unique integers, meaning no two elements $X[i]$ and $X[j]$ are equal for $1 \leq i \neq j \leq N$.

- Design a CRCW PRAM algorithm $\text{SORT}(X[1 \dots N])$ with N^2 PUs, running in $O(1)$ time, that returns the sorted array $X'[1 \dots N]$, such that $X'[i] < X'[i+1]$ for all i . Specify the concurrency mode used for the concurrent writes.
- What are its time, work and cost?
- How can you make the algorithm work if the elements of $X[1 \dots N]$ are not unique?
- How can you make the algorithm work on a CREW PRAM, still with N^2 PUs? What are its time, work and cost?

Solution

(a) Sorting by *rank*: the position of $X[i]$ in the sorted array is $1 + r[i]$, where $r[i]$ is the number of elements smaller than $X[i]$. With N^2 PUs, one per pair (i, j) , all the comparisons are done at once, and the ranks are obtained by counting with a concurrent write.

Line 4 is a concurrent write in **fusion mode with** $+$, as SUMARRAYCRCW in the lecture: $r[i]$ receives the number of PUs (i, j) that wrote, i.e., the number of j with $X[j] < X[i]$ (and keeps 0 if none wrote). The reads are concurrent too: $X[i]$ is read by the $2N$ PUs (i, j) and (j, i) . The writes of line 6 are exclusive: the elements being unique, the ranks $r[1], \dots, r[N]$ are a permutation of $0, \dots, N-1$.

$\text{SORT}(X[1 \dots N])$

```

1: forall  $i = 1, \dots, N$  in parallel do
2:    $r[i] \leftarrow 0$ 
3:   forall  $(i, j) \in \{1, \dots, N\}^2$  in parallel do
4:     if  $X[j] < X[i]$  then  $r[i] \leftarrow 1$            ► fusion +
5:   forall  $i = 1, \dots, N$  in parallel do
6:      $X'[r[i] + 1] \leftarrow X[i]$ 
7:   return  $X'[1 \dots N]$ 

```

(b) Three loops of $O(1)$ time: $T(N) = O(1)$ with N^2 PUs, $C(N) = O(N^2)$, and $W(N) = O(N^2)$ since all the N^2 PUs compare in line 4. With $T_{\text{seq}}(N) = O(N \log N)$: neither work-optimal nor efficient.

(c) Two equal elements get the same rank, hence the same position, and one position stays empty. Break the ties by the index: count j such that $X[j] < X[i]$ **or** $(X[j] = X[i] \text{ and } j < i)$. The ranks are again a permutation of $0, \dots, N-1$, and equal elements keep their original order (a stable sort).

(d) Without concurrent writes, the counting of line 4 is replaced by a reduction: PU (i, j) writes its comparison bit into its own cell $c[i][j]$, then $r[i] \leftarrow \text{REDUCE}(c[i][1 \dots N], +)$ for all i in parallel, N reductions with N PUs each, in $O(\log N)$ time. The reads of X stay concurrent: CREW. Time $O(\log N)$ with N^2 PUs, cost $O(N^2 \log N)$, work $O(N^2)$ (N^2 comparisons and N reductions of $O(N)$ work each).

Exercise 10

Parallel matrix multiplication

Let $A[1 \dots N][1 \dots N]$ and $B[1 \dots N][1 \dots N]$ be two square matrices. Their product is the square matrix $C = AB$ whose elements are defined as follows:

$$C[i][j] = \sum_{k=1}^N A[i][k] B[k][j], \quad 1 \leq i, j \leq N$$

- Design a CRCW PRAM algorithm $\text{MATMUL}(A, B)$ with N^3 PUs, running in $O(1)$ time, that carries out the multiplication of two $N \times N$ matrices $C = AB$, then returns the matrix C . What are its time, work and cost?
- Do the same for a CREW PRAM, still with N^3 PUs. What are its time, work and cost? How can you make this algorithm work on an EREW PRAM as well?

Solution

(a) One PU per product $A[i][k]B[k][j]$, i.e., N^3 PUs, and the N products of each $C[i][j]$ are summed by a concurrent write.

Line 4 is a concurrent write in **fusion mode with +**: the N PUs $(i, j, 1), \dots, (i, j, N)$ write the cell $C[i][j]$, which receives the sum of the written values. The reads are concurrent as well: $A[i][k]$ is read by the N PUs $(i, 1, k), \dots, (i, N, k)$. Time $O(1)$, with N^3 PUs; work $O(N^3)$ (one product per PU) and cost $O(N^3)$. With respect to the standard sequential algorithm, $T_{seq}(N) = O(N^3)$: work-optimal and efficient (faster sequential algorithms exist, such as Strassen's, which we ignore here).

(b) Without concurrent writes, each PU stores its product in a cell of its own, and the N products of each $C[i][j]$ are summed with a reduction: N^2 reductions in parallel, with N PUs each.

Time $O(1)$ for line 2 and $O(\log N)$ for line 4: $T(N) = O(\log N)$ with N^3 PUs, cost $C(N) = O(N^3 \log N)$, work $W(N) = O(N^3)$ (N^3 products, and N^2 reductions of $O(N)$ work each). Work-optimal, not efficient.

MATMUL(A, B)

```

1: forall  $(i, j) \in \{1, \dots, N\}^2$  in parallel do
2:    $C[i][j] \leftarrow 0$ 
3:   forall  $(i, j, k) \in \{1, \dots, N\}^3$  in parallel do
4:      $C[i][j] \leftarrow A[i][k] \cdot B[k][j]$  ► fusion +
5:   return  $C$ 

```

MATMULCREW(A, B)

```

1: forall  $(i, j, k) \in \{1, \dots, N\}^3$  in parallel do
2:    $P[i][j][k] \leftarrow A[i][k] \cdot B[k][j]$ 
3:   forall  $(i, j) \in \{1, \dots, N\}^2$  in parallel do
4:      $C[i][j] \leftarrow \text{REDUCE}(P[i][j][1 \dots N], +)$  ►  $N$  PUs each
5:   return  $C$ 

```

EREW: the only concurrent accesses left are the reads of $A[i][k]$ by the N PUs (i, j, k) , $j = 1 \dots N$, and of $B[k][j]$ by the N PUs (i, j, k) , $i = 1 \dots N$. Replace them by N copies, made by BCAST: $pa[i][k][1 \dots N] \leftarrow \text{BCAST}(A[i][k], N)$ for all (i, k) in parallel, and $pb[k][j][1 \dots N] \leftarrow \text{BCAST}(B[k][j], N)$ for all (k, j) in parallel, i.e., $2N^2$ broadcasts with N PUs each, in $O(\log N)$ time and $O(N^3)$ work. Then PU (i, j, k) reads its own copies $pa[i][k][j]$ and $pb[k][j][i]$ in line 2. The time stays $O(\log N)$, the work $O(N^3)$ and the cost $O(N^3 \log N)$, at the price of $O(N^3)$ memory.

Exercise 11

Simulation of concurrency modes on a CRCW PRAM

Consider a CRCW PRAM in *consistent* mode: simultaneous writes to the same memory cell are allowed only if they all write the same value.

- a) Prove that a CRCW PRAM in consistent mode can simulate a CRCW PRAM in *fusion mode with min* (a cell written simultaneously by several PUs receives the minimum of the written values), with the same time up to a constant factor. The simulation may use more PUs than the simulated algorithm. Hint: with k^2 PUs, compare all the pairs among the k values written to a cell, and mark every value that is larger than another one; which concurrency mode does the marking need?

Solution

Let the simulated algorithm use P PUs; the simulating machine uses P^2 PUs, indexed by the pairs (i, j) , and $3P$ extra cells $addr[1 \dots P]$, $val[1 \dots P]$, $mark[1 \dots P]$. Every step of the simulated algorithm that is not a write is executed as is, by the PUs (i, i) . A write step, in which PU i writes the value v_i to the address a_i , is replaced by the three steps below, so that the time is multiplied by a constant.

Line 2: exclusive writes (each PU its own cells). The PUs (i, i) execute lines 2 and 6, the PUs (i, j) line 4. Line 4: the PUs (i, j) with $a_j = a_i$ and $v_j < v_i$ all write the *same* value, **true**, into $mark[i]$: the **consistent** mode is exactly what is needed (the reads of lines 4 and 6 are concurrent, which is fine on a CRCW PRAM). After line 4, among the PUs that write a given cell a , the unmarked ones are exactly those whose value is the minimum m_a of the values written to a . Line 6: the unmarked PUs write to a , and they all write the same value m_a : consistent mode again, and a receives min of the written values, as in fusion mode with min.

Hence a consistent-mode CRCW PRAM with P^2 PUs simulates a fusion-mode-with-min CRCW PRAM with P PUs, in at most three times the time (a constant factor). The same construction simulates the priority mode (mark i in line 4 when $a_j = a_i$ and $j < i$) and fusion with max (mark when $v_j > v_i$).

SIMULATED WRITE STEP

```

1: forall  $i = 1, \dots, P$  in parallel do
2:    $addr[i] \leftarrow a_i$ ;  $val[i] \leftarrow v_i$ ;  $mark[i] \leftarrow \text{false}$ 
3:   forall  $(i, j) \in \{1, \dots, P\}^2$  in parallel do
4:     if  $addr[j] = addr[i]$  and  $val[j] < val[i]$  then  $mark[i] \leftarrow \text{true}$ 
5:   forall  $i = 1, \dots, P$  in parallel do
6:     if  $mark[i] = \text{false}$  then STORE( $addr[i]$ ,  $val[i]$ ) ► consistent

```

Exercise 12

Computing the depth of nodes in a tree

Let $parent[1 \dots N]$ be the array of parent pointers in a tree having N nodes (such that $parent[i] = \text{NIL}$ if the node i is the root of the tree, and $1 \leq parent[i] \neq i \leq N$ otherwise). The goal of this exercise is to compute the depth of all nodes in the tree in parallel. The depth of the root node is defined to be zero, and the depth of any other node is the depth of its parent plus one (i.e., $depth[i] = depth[parent[i]] + 1$).

- Design a CREW PRAM algorithm $TREEDEPTH(parent[1 \dots N])$ with N PUs to compute the depth of each node in the tree, and return it in an array. What are its time, work and cost? Would your algorithm work on an EREW PRAM?
- This time, assume that the tree is binary, and in addition to parent pointers, each node i also has a pointer towards its left and right children $left[i]$ and $right[i]$ (with $left[i] = \text{NIL}$ if the node i has no left child, likewise for the right child). Design an EREW PRAM algorithm $BINTREEDEPTH(parent[1 \dots N], left[1 \dots N], right[1 \dots N])$ with $O(N)$ PUs to compute and return the depth of all nodes in the tree in $O(\log_2 N)$ time. Hint: Consider the depth-first traversal of the tree, and try to use the prefix scan algorithm on this traversal.

Solution

(a) Pointer jumping on the parent pointers, exactly as list ranking: the path from a node to the root is a linked list, and the depth of the node is its rank in this list.

Invariant: $depth[i]$ is the distance from i to $p[i]$ (to the root if $p[i] = \text{NIL}$); after k iterations, $p[i]$ is the ancestor of i at distance 2^k , or NIL . The depth of a node is at most $N - 1$, so $\lceil \log_2 N \rceil$ iterations are enough, and the loop can be run a fixed number of times: no termination test, hence no flag to compute. Time $O(\log N)$ with N PUs; the N PUs are created at each of the $\lceil \log_2 N \rceil$ iterations: work $O(N \log N)$ and cost $O(N \log N)$. With $T_{seq}(N) = O(N)$ (a traversal of the tree): neither work-optimal nor efficient.

EREW: no. The writes are exclusive (every PU writes its own cells), but $parent$ is not injective: in lines 7–8, all the children of a node j read $depth[j]$ and $p[j]$ at the same step, and after k iterations up to 2^k nodes share the same $p[i]$. The algorithm needs concurrent reads (unless the tree is a path).

(b) The depth-first traversal of the tree (left subtree first) goes down and up along the edges: it is a sequence of $2(N - 1)$ arcs, every edge being traversed once downwards, then once upwards. Give the weight $+1$ to the downward arcs and -1 to the upward arcs: after any prefix of the traversal, the sum of the weights is the depth of the current node. The depth of v is therefore the prefix sum of the traversal up to the arc that enters v , and the traversal is a linked list of arcs on which Exercise 6 computes all these prefix sums at once.

For every node v , name d_v the arc that enters v from its parent and u_v the arc that goes back from v to its parent (numbered $d_v = v$ and $u_v = N + v$ in arrays of $2N$ cells; for the root, d_r is a fictitious first arc of weight 0, and u_r is unused). The successor of each arc is given by the depth-first order:

- after entering v (arc d_v), the traversal enters the left child of v if it exists, otherwise its right child, otherwise it goes back up (arc u_v);
- after going back up from v to p (arc u_v), it enters the right child of p if v is the left child of p and p has a right child, otherwise it goes back up from p (arc u_p), or stops if p is the root.

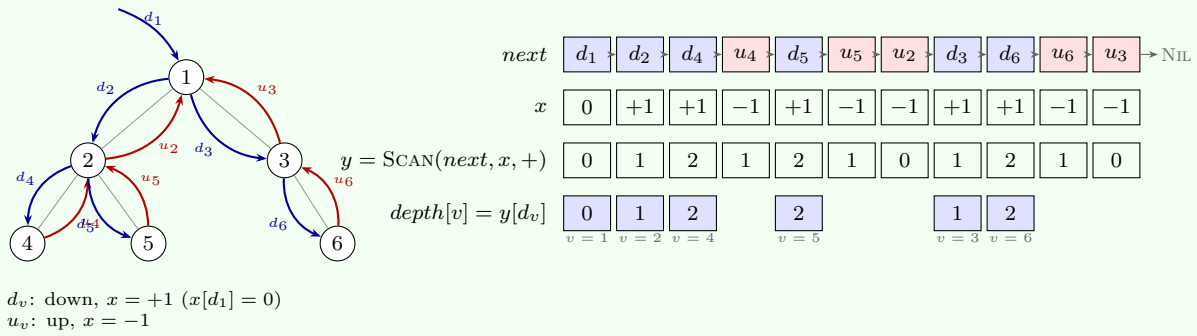
The first rule only needs the cells of v ; the second one needs the cells of p , which both children of p would read: instead, every node p computes the successors of the up arcs of its children (lines 8–12), with its own cells only. On the tree below, the traversal is the list $d_1 \rightarrow d_2 \rightarrow d_4 \rightarrow u_4 \rightarrow d_5 \rightarrow u_5 \rightarrow u_2 \rightarrow d_3 \rightarrow d_6 \rightarrow u_6 \rightarrow u_3$, and the prefix sums of its weights at the down arcs are the depths.

 $TREEDEPTH(parent[1 \dots N])$

```

1: forall  $i = 1, \dots, N$  in parallel do
2:   if  $parent[i] = \text{NIL}$  then  $depth[i] \leftarrow 0$  else 1
3:    $p[i] \leftarrow parent[i]$ 
4:   for  $k = 1, \dots, \lceil \log_2 N \rceil$  do
5:     forall  $i = 1, \dots, N$  in parallel do
6:       if  $p[i] \neq \text{NIL}$  then
7:          $depth[i] \leftarrow depth[i] + depth[p[i]]$ 
8:          $p[i] \leftarrow p[p[i]]$ 
9:   return  $depth[1 \dots N]$ 

```



Exclusivity: in lines 2–6, PU v reads its own cells $\text{parent}[v]$, $\text{left}[v]$, $\text{right}[v]$ and writes its own arcs d_v , u_v ; in lines 8–12, PU v reads its own cells and writes the arcs $u_{\text{left}[v]}$ and $u_{\text{right}[v]}$, which no other PU writes since a node has one parent; up is a private register. The traversal visits every arc once, so next is injective and SCAN runs on an EREW PRAM (Exercise 6(b)); $+$ is commutative, so the order of the operands does not matter. Line 13 computes $y[a]$, the sum of the weights from the head d_r of the traversal to the arc a (Exercise 6(a)); $y[d_v]$ is the depth of v , and $y[d_r] = 0$.

Time: lines 1–12 and 14–16 take $O(1)$, line 13 takes $O(\log N)$ with $2N$ PUs: $T(N) = O(\log N)$, with $W(N) = O(N \log N)$ and $C(N) = O(N \log N)$, the profile of SCAN .

BINTREEDEPTH($\text{parent}[1 \dots N]$, $\text{left}[1 \dots N]$, $\text{right}[1 \dots N]$)

```

1: forall v = 1, ..., N in parallel do
2:   if parent[v] = NIL then x[d_v] ← 0 else x[d_v] ← 1
3:   x[u_v] ← -1; next[u_v] ← NIL
4:   if left[v] ≠ NIL then next[d_v] ← d_left[v]
5:   else if right[v] ≠ NIL then next[d_v] ← d_right[v]
6:   else next[d_v] ← u_v
7: forall v = 1, ..., N in parallel do
8:   if parent[v] = NIL then up ← NIL else up ← u_v
9:   if left[v] ≠ NIL then
10:    if right[v] ≠ NIL then next[u_left[v]] ← d_right[v]
11:    else next[u_left[v]] ← up
12:   if right[v] ≠ NIL then next[u_right[v]] ← up
13: y[1 ... 2N] ← SCAN(next[1 ... 2N], x[1 ... 2N], +)
14: forall v = 1, ..., N in parallel do
15:   depth[v] ← y[d_v]
16: return depth[1 ... N]

```

► leaf: back up

► Exercise 6

Exercise 13

Parallel breadth-first search

The goal of this exercise is to perform a parallel breadth-first search (BFS) on a given directed graph $G(V, E)$ having $|V|$ vertices and $|E|$ edges $E[t] = (u_t, v_t)$ for $1 \leq t \leq |E|$, with $1 \leq u_t \neq v_t \leq |V|$. The graph is strongly connected: every vertex is reachable from every other one. Its *diameter* $\delta(G)$ is the largest distance between two vertices.

- Design a CRCW PRAM algorithm $\text{BFS}(G(V, E), s)$ with $|E|$ PUs, one per edge, that performs a parallel BFS on G , starting from the vertex s ($1 \leq s \leq |V|$). The algorithm should compute and return two arrays: $\text{parent}[i]$, the parent of each vertex i in the BFS tree, and $\text{level}[i]$, the level of vertex i in the BFS tree (i.e., its distance from the starting vertex s). Specify the concurrency mode to be employed.
- What are its time, work and cost, as functions of $|V|$, $|E|$ and $\delta(G)$?

Solution

(a) Level by level: at iteration k , every edge (u, v) whose tail u is at level k and whose head v is not visited yet puts v at level $k + 1$, with u as parent. One PU per edge.

Concurrency: several edges $(u, v), (u', v)$ with u, u' at level k may write v at the same step. Line 8 writes the same value $k + 1$ in every case: **consistent** mode. Line 9 writes different values, u or u' : **arbitrary** mode, any of the writers is a valid parent in a BFS tree since all of them are at level k (the **priority** mode gives a deterministic choice). Line 10: consistent mode (every writer writes **true**). The reads are concurrent too: $level[u]$ is read by all the edges leaving u . All the reads of an iteration happen before its writes, so the test of line 7 sees the levels of the previous iteration.

BFS($G(V, E), s$)

```

1: forall  $i = 1, \dots, |V|$  in parallel do
2:    $level[i] \leftarrow \infty$ ;  $parent[i] \leftarrow \text{NIL}$ 
3:  $level[s] \leftarrow 0$ ;  $k \leftarrow 0$ ;  $changed \leftarrow \text{true}$ 
4: while  $changed$  do
5:    $changed \leftarrow \text{false}$ 
6:   forall  $t = 1, \dots, |E|$  in parallel do
7:     if  $level[u_t] = k$  and  $level[v_t] = \infty$  then
8:        $level[v_t] \leftarrow k + 1$            ▶ consistent
9:        $parent[v_t] \leftarrow u_t$          ▶ arbitrary
10:       $changed \leftarrow \text{true}$           ▶ consistent
11:    $k \leftarrow k + 1$ 
12: return  $parent[1 \dots |V|], level[1 \dots |V|]$ 

```

(b) The loop runs $L + 1$ times, where L is the largest level, i.e., the largest distance from s to another vertex; $L \leq \delta(G)$, and $L = \delta(G)$ for the vertex s farthest from the others. Every iteration takes $O(1)$ time: $T = O(\delta(G))$ with $|E|$ PUs (the initialization of the $|V|$ vertices also fits in $|E|$ PUs, as $|E| \geq |V|$ in a strongly connected graph). Cost $C = O(\delta(G) \cdot |E|)$. Work: the $|E|$ PUs are created at every iteration, and each of them evaluates its test, even though an edge (u, v) does something in at most one iteration (the one in which u gets its level): $W = O(|V| + \delta(G) \cdot |E|)$. With $T_{seq} = O(|V| + |E|)$ (sequential BFS): neither work-optimal nor efficient, unless $\delta(G) = O(1)$. Creating only the PUs of the edges leaving the current level would require gathering these edges first, which is not $O(1)$.

Exercise 14

Connected components of an undirected graph

Let $G(V, E)$ be an undirected graph with $|V|$ vertices and $|E|$ edges $E[t] = (u_t, v_t)$ for $1 \leq t \leq |E|$, with $1 \leq u_t \neq v_t \leq |V|$. We want to label every vertex i with $C[i]$, the smallest vertex index of the connected component that i belongs to. We will use a simple algorithm: start with $C[i] = i$ for every vertex, then let every edge *merge* its two ends in parallel, by giving both of them the smaller of their two labels; repeat until no edge changes anything, i.e., until both ends of every edge carry the same label.

- Design a CRCW PRAM algorithm **CONNECTEDCOMPONENTS**(V, E) that follows this idea, with $|E|$ PUs, one per edge. Specify the concurrency modes: for the writes to the labels, and for the test that stops the loop.
- Show that the algorithm terminates and that the labels are correct in the end. Hint: let m be the smallest vertex of a component; after k iterations, which vertices of the component are guaranteed to carry the label m ?
- What are its time, work and cost, as functions of $|V|$, $|E|$ and the largest diameter D of a component (the diameter is the largest distance between two vertices of the component)? Is it work-optimal? Is it efficient?
- Give a family of graphs with $|V|$ vertices on which the algorithm needs $\Theta(|V|)$ iterations. (The optimal $O(\log |V|)$ algorithms for this problem combine this idea with pointer jumping on the labels; they are beyond the scope of this course.)

Solution

(a) One PU per edge; at every iteration, every edge whose two ends carry different labels gives the smaller label to both ends.

Writes to the labels (line 9): a vertex v with several neighbours receives several values at once, one per edge, and must keep the smallest: **fusion mode with min** (a vertex may also receive its own label again, which is harmless). With the arbitrary mode the algorithm would still terminate (a label only decreases), but the bound of (c) would not hold. Stop test (line 10): every edge that changes something writes **true**: **consistent** mode (or fusion with **or**). The reads of $C[u_t]$, $C[v_t]$ are concurrent as well.

CONNECTEDCOMPONENTS(V, E)

```

1: forall  $i = 1, \dots, |V|$  in parallel do
2:    $C[i] \leftarrow i$ 
3:  $changed \leftarrow \text{true}$ 
4: while  $changed$  do
5:    $changed \leftarrow \text{false}$ 
6:   forall  $t = 1, \dots, |E|$  in parallel do
7:     if  $C[u_t] \neq C[v_t]$  then
8:        $m \leftarrow \min(C[u_t], C[v_t])$ 
9:        $C[u_t] \leftarrow m$ ;  $C[v_t] \leftarrow m$            ▶ fusion min
10:       $changed \leftarrow \text{true}$                        ▶ consistent
11: return  $C[1 \dots |V|]$ 

```

(b) Labels are only copied from a neighbour, so the label of a vertex is always the index of a vertex of its own component, and it never increases. Let K be a component and m its smallest vertex: $C[m] = m$ forever (no

smaller label exists in K). Claim: after k iterations, every vertex of K at distance at most k from m carries the label m . By induction on k : true for $k = 0$; if v is at distance $k + 1$ from m , it has a neighbour w at distance k , which carries m after k iterations; at iteration $k + 1$, the edge (w, v) writes $\min(m, C[v]) = m$ to $C[v]$, and the fusion with min cannot give less than m : $C[v] = m$. Since every vertex of K is at distance at most D from m , after D iterations all the vertices of K carry m ; the next iteration finds the two ends of every edge equal, writes nothing and stops the loop. At the end, $C[i]$ is the smallest vertex of the component of i .

(c) At most $D + 1$ iterations of $O(1)$ time: $T = O(D + 1)$, with $|E|$ PUs for the loop ($\max(|V|, |E|)$ in total); cost $C = O((D + 1)(|V| + |E|))$. Work: the $|E|$ PUs are created at every iteration (each evaluates its test, and most of them have something to do during many iterations): $W = O(|V| + D \cdot |E|)$. With $T_{seq} = O(|V| + |E|)$ (one BFS or DFS): neither work-optimal nor efficient, unless $D = O(1)$.

(d) A path $1 - 2 - \dots - |V|$ has diameter $|V| - 1$, and by (b) the label 1 advances by exactly one vertex per iteration: $|V| - 1$ iterations before the last vertex gets it, plus one to detect the end ($\Theta(|V|)$ iterations; a cycle behaves likewise).